Конспект лекции 5:
Основы обработки естественного языка

Липкин С.М.

1 Что такое NLP?

Обработка естественного языка (Natural Language Processing, NLP) — область на стыке искусственного интеллекта, лингвистики и информатики, изучающая методы взаимодействия компьютера и человеческого языка.

История развития: зародилась в 1950-х годах (тест Тьюринга). Ранние подходы использовали правила и статистику. Современный этап (2013–настоящее время) характеризуется доминированием глубокого обучения: word2vec, seq2seq, Transformer, LLM.

Базовый конвейер NLP:

$$\text{Текст} \xrightarrow{\text{Предобработка}} \text{Токены} \xrightarrow{\text{Векторизация}} \text{Эмбеддинги} \xrightarrow{\text{Модель}} \text{Ответ}$$

Области применения: чат-боты, поиск, машинный перевод, анализ тональности, голосовые помощники, аннотация текстов, вопросно-ответные системы.

2 Токенизация

Токенизация — разбиение текста на минимальные осмысленные единицы (токены). Это обязательный первый шаг NLP-конвейера.

2.1 Основные подходы

1. **Word-level:** разделение по пробелам и знакам пунктуации. Даёт большой словарь (10^5–10^6 токенов). Проблема: OOV (out-of-vocabulary) — модель не знает редких слов.

2. **Subword (подсловный):** разбиение на подслова. Баланс размера словаря и покрытия. Решает OOV. **BPE (Byte-Pair Encoding)** — итеративное слияние частых пар символов, используется в GPT. **WordPiece** — слияние по максимизации likelihood, используется в BERT.

3. **Character-level:** словарь состоит из символов алфавита (30–200 токенов). Минимальный размер словаря, но длинные последовательности (каждое слово — несколько символов), сложнее улавливать смысл.

3 Нормализация текста

Набор шагов для приведения текста к единообразному виду:

3.1 Базовые шаги

1. **Нижний регистр:** «Москва» и «москва» — один токен. Уменьшает словарь.
2. **Удаление стоп-слов:** предлоги, союзы, местоимения несут мало смысла и удаляются (актуально не для всех задач — для некоторых, наоборот, важны).
3. **Очистка от спецсимволов:** удаление пунктуации, URL, HTML-тегов, цифрового мусора.

3.2 Стемминг и лемматизация

Стемминг: механическое отсечение окончаний по правилам. Быстро, но грубо. Без учёта контекста. Пример: «бегущий» → «бегущ», «красивее» → «красив». Алгоритмы: Портера, Snowball. Есть версии для русского языка.

Лемматизация: лингвистически корректное приведение к начальной форме (лемме) через словарь и морфологический анализ. Точнее, но медленнее стемминга. Пример: «бежал» → «бежать», «красивее» → «красивый». Инструменты: NLTK, spaCy, pymorphy2 (русский).

4 Типовые задачи NLP

1. **Классификация текста:** определение категории (спам, тематика, жанр).
2. **Анализ тональности:** определение эмоциональной окраски (позитив/негатив/нейтрально).
3. **Машинный перевод:** автоматический перевод между языками.
4. **NER (Named Entity Recognition):** извлечение именованных сущностей (имена, даты, организации, локации).
5. **Генерация текста:** создание связного текста (саммаризация, диалоги).
6. **Вопросно-ответные системы (QA):** поиск ответа на вопрос в заданном контексте.
7. **Языковое моделирование:** предсказание следующего слова.
8. **Парсинг:** синтаксический анализ, dependency parsing.
9. **Textual Entailment:** определение логической связи между текстами.

5 RNN в NLP

5.1 Принцип работы

RNN обрабатывают элементы последовательности по одному, сохраняя скрытое состояние (память), которое передаётся на каждом шаге:

$$h_t = f(Wh_{t-1} + Ux_t + b)$$

Язык — последовательная структура, поэтому RNN хорошо подходят: учитывают порядок слов и могут моделировать контекст.

5.2 Проблемы RNN

При длинных последовательностях базовая RNN теряет информацию о ранних элементах из-за многократного умножения градиентов (исчезающий градиент).

5.3 LSTM (Long Short-Term Memory)

Hochreiter & Schmidhuber, 1997. Решает проблему через gating-механизм и отдельную ячейку состояния (cell state):

$$
\begin{aligned}
f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) \quad \text{(forget gate)} \\
i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) \quad \text{(input gate)} \\
\tilde{C}_t &= \tanh(W_C[h_{t-1}, x_t] + b_C) \quad \text{(кандидат)} \\
C_t &= f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \\
o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) \quad \text{(output gate)} \\
h_t &= o_t \odot \tanh(C_t)
\end{aligned}
$$

Cell state — «конвейер» информации с минимальными изменениями. Forget gate решает, что забыть, input gate — что добавить, output — что вывести.

5.4 GRU (Gated Recurrent Unit)

Упрощённая версия LSTM с двумя gate:

$$
\begin{aligned}
r_t &= \sigma(W_r[h_{t-1}, x_t] + b_r) \quad \text{(reset gate)} \\
z_t &= \sigma(W_z[h_{t-1}, x_t] + b_z) \quad \text{(update gate)} \\
\tilde{h}_t &= \tanh(W[r_t \odot h_{t-1}, x_t] + b) \\
h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t
\end{aligned}
$$

Меньше параметров, быстрее обучение, меньше памяти, сопоставимое качество.

5.5 LSTM vs GRU

- LSTM: 3 gate, отдельный cell state, больше параметров, лучше на длинных последовательностях.
- GRU: 2 gate, единое скрытое состояние, меньше параметров, быстрее, сопоставимое качество на коротких текстах.

6 Seq2Seq и механизм внимания

6.1 Encoder-Decoder

Архитектура для задач, где вход и выход — последовательности разной длины (перевод, саммаризация):

- **Encoder:** RNN/LSTM читает входную последовательность и генерирует последовательность скрытых состояний $h_1, \ldots, h_T$.
- **Decoder:** RNN/LSTM генерирует выходную последовательность, используя контекст от энкодера.

Проблема: при длинных предложениях информация теряется при «упаковке» в один вектор контекста.

6.2 Механизм внимания (Bahdanau, 2015)

На каждом шаге декодер вычисляет веса α_{tj} для всех состояний энкодера:

$$e_{tj} = a(s_{t-1}, h_j), \quad \alpha_{tj} = \frac{\exp(e_{tj})}{\sum_k \exp(e_{tk})}, \quad c_t = \sum_{j=1}^{T} \alpha_{tj} h_j$$

Контекстный вектор c_t — взвешенная сумма состояний энкодера с акцентом на релевантные части.

Значение: внимание стало фундаментом архитектуры Transformer (Vaswani et al., 2017), который лежит в основе GPT, BERT и всех современных LLM.

7 Применение RNN в NLP

7.1 Классификация текста и тональность

RNN читает текст слово за словом, накапливая контекст в скрытом состоянии. Последнее скрытое состояние (или max-pooling по всем шагам) подаётся на классификатор (Dense + Softmax).

Пример: отзыв «Этот фильм великолепен!» проходит через 4 шага LSTM, финальное состояние кодирует положительную тональность, Softmax выдаёт P(позитив) = 0.92.

BiLSTM: двунаправленный LSTM обрабатывает текст слева направо и справа налево, захватывая полный контекст.

7.2 NER: BiLSTM-CRF

Стандартная архитектура для NER:

- **BiLSTM** извлекает контекстные признаки для каждого токена (с учётом контекста слева и справа).
- **CRF-слой** (Conditional Random Field) обеспечивает согласованность меток: например, B-PER не может идти после I-ORG; I-ORG может идти только после B-ORG.

BIO-тегирование: B (begin) — начало сущности, I (inside) — продолжение, O (outside) — не сущность. Типы: PER (персона), ORG (организация), LOC (локация), DATE (дата).

Пример: «Иван работает в Газпром с 2010 года» → B-PER, O, O, B-ORG, O, DATE, O.

7.3 Машинный перевод

Seq2Seq с вниманием — стандарт перевода до появления Transformer. Энкодер читает исходное предложение, декодер генерирует перевод. На каждом шаге внимание сфокусировано на релевантных частях исходного предложения.

Пример: при переводе «Я изучаю нейронные сети» на шаге «study» максимальный вес attention направлен на «изучаю».

7.4 Языковое моделирование и генерация

Языковая модель предсказывает вероятность последовательности:

$$P(w_1, \ldots, w_T) = \prod_{t=1}^{T} P(w_t | w_{<t})$$

На этапе генерации модель работает **авторегрессивно**: на каждом шаге подаёт собственный выход как следующий вход.

Стратегии декодирования:

- Greedy (argmax): выбирается самый вероятный токен.
- Temperature sampling: $P(y) = \text{softmax}(\text{logits}/\tau)$. $\tau \to 0$ — greedy, $\tau \to \infty$ — равномерное.
- Top-K sampling: семплинг из K лучших токенов.
- Top-P (nucleus) sampling: семплинг из минимального набора токенов, покрывающих массу вероятности p.

8 Сравнение подходов

Критерий	**RNN**	**LSTM/GRU**	**Transformer**
Длинные зависимости	Слабо	Хорошо	Отлично
Параллелизация	Нет	Нет	Да
Скорость обучения	Быстрая	Средняя	Высокая (GPU)
Качество (SOTA)	Низкое	Среднее	SOTA
Потребление памяти	Мало	Средне	Много
Типичное применение	Простые задачи	Перевод, NER	LLM, чат-боты

9 Заключение

NLP прошёл путь от правил и статистики до глубокого обучения. RNN/LSTM/GRU остаются актуальны для задач с ограниченными ресурсами, streaming-данных и real-time обработки. Transformer доминирует в качестве, но требует больше вычислительных ресурсов.

Ключевые этапы конвейера — токенизация и нормализация — критически важны для качества любой NLP-модели. Выбор подхода (word, subword, character) зависит от языка, задачи и доступных ресурсов.